CSE235 Week 9: Dictionaries

Lists in Python are indexed by integers starting at 0. A dictionary is like a list, but it stores key-value pairs, allowing the "keys" to be used as subscripts. These are similar to associative arrays or maps in other languages. First, we'll look at sets, which are the other built-in container type in Python.

Set

A set is a collection where all its elements are unique. A set can be constructed like a list or tuple, except the set uses curly braces `{}` instead of square brackets. An empty set cannot be created using `empty_set = {}`, however, since curly braces are also used for dictionaries, and take precedence for dictionaries.

```
# create an empty set
set1 = set()
# create a set with strings
set2 = { "a", "cat", "apple", "car" }
# create a set with mixed elements
set3 = { 1, "fish", 2.0, "fish", "red", "fish", True, "fish" }
# create a set using a set comprehension
set4 = { x for x in range(10) }
# create a set from a list
set5 = set([])
```

When creating a set, any duplicate items will automatically be removed from the set. For example:

```
no_duplicates = { "apple", "orange", "apple", "banana", "orange" }
print(no_duplicates)
# prints: {'banana', 'orange', 'apple'}
# note that sets do not preserve order
```

Any iterable collection can be converted to a set (anything that supports

the `for x in y` syntax). Even a string can be used to create a set:

```
set_from_str = set("This is an example of creating a set from a string
print(set_from_str)
# prints:
# {'r','x','t','a','n',' ','f','g','s','h','m','c','i','o','p','e','T'
```

New elements can be added to the set with the `add()` function and element can be removed with the `remove()` function. `remove()` will throw a `KeyError` exception if the element is not present. The `discard()` function is a version of `remove()` that does not throw an exception if the value is not present.

```
fruits = {"apple", "banana", "grape", "orange"}
fruits.add("pear")

try:
    fruits.remove("cherry")
except KeyError as ex:
    print(f"Key Error: {ex}")
    # prints: Key Error: 'cherry'

# this is fine, no exception is thrown
# even though cherry is not present in the set
fruits.discard("cherry")
```

Sets are primarily used to check if items are present in a collection. There is no way to access the individual elements of a set.

```
fruits = {"apple", "banana", "grape", "orange"}
test_value = input("Enter a fruit: ")
# check if the input value is in the set using the `in` operator
if test_value in fruits:
    print(f"{} is in the fruit set")
else:
    print(f"{} is not in the fruit set")
```

Set elements can be looped through like a list:

```
fruits = {"apple", "banana", "grape", "orange"}
for fruit in fruits:
    print(fruit)
```

Output:

```
orange
grape
apple
banana
```

All elements of a set must be "hashable". Python's data types that are not modified when passed to a function (like int, float, str, bool) are hashable. Classes (discussed next week) can have hashes implemented for them, as well. The specific details of implementing a good hash function are beyond the scope of this course.

Dictionaries

Dictionaries store "key, value" pairs. The keys of a dictionary are unique, and like set elements, must be hashable. A dictionary can be created using the following syntax:

```
# create an empty dictionary
dict1 = {}
# create an empty dictionary
dict2 = dict()
# create a dictionary using {}
dict3 = { "key1": 0, "key2", 1}
# create a dictionary using dict()
dict4 = dict(("key1", 0), ("key2", 1))
```

Elements of a dictionary can be accessed by passing its key to the subscript operator:

```
student = { "Name": "John Doe", "GPA": 3.5 }
print(f"Name: {student["Name"]}")
print(f"GPA:  {student["GPA"]}")
```

The subscript operator can also be used to modify an existing key's value, or add a key to the dictionary:

```
student = { "Name": "John Doe", "GPA": 3.5 }
student["Credit Hours"] = 90
```

Attempting to read a key that does not exist in a dictionary raises a

`KeyError`. The `get()` function can be used to try to read a key's value. If the key is not present, it will return `None` instead of raising an exception.

```
student = { "Name": "John Doe", "GPA": 3.5 }
student["Credit Hours"] = 90
# raises a key error
grade_level = student["Level"]
# grade_level is None
grade_level = student.get("Level")
```

The `keys()` member function returns a view of the dictionary's keys. A view is what the `range()` function returns. It can be converted to a list using the `list()` function, or used directly in a for loop:

```
# create a list based on the keys
list_of_keys = list(student.keys())

# print the keys of the dictionary
for key in student.keys():
    print(key)
```

The `values()` function returns a view of the dictionary's values:

```
# print the values in the dictionary
for value in student.values():
    print(value)
```

The `items()` function returns a view of tuples with each (key, value) pair. They can be unpacked like the following for-loop shows

```
# print the keys and values in the dictionary
for key, value in student.items():
    print(f"{key}:\t{value}")
```

Items are removed from a dictionary using the `del` operator or the `pop()` function. The `popitem()` function removes the (key, value) pair and returns it. A `KeyError` is raised if the key is not present.

```
del student["Name"]
student.pop("GPA")
key, value = student.popitem("Credit Hours")
```

Dictionaries are used to store data. In the next section, we'll look at JSON, which is a data format that is easily represented by a dictionary. The rows of a SQL table can also be represented by a dictionary, along with CSV data. Dictionaries can be used any time the data being stored has a label that makes access easier (compared to a list, which only uses indices to access individual elements).

JSON

JSON is an acronym for JavaScript Object Notation. It is a popular data format, especially for the web, but has a wide variety of applications beyond that as well. Python has a built-in module for handling JSON files, called `json`. JSON contains two top-level entities:

- Objects
  - Enclosed in curly braces `{}`
  - Keys are raw text, but converted to strings when parsed
  - Values: strings, numbers, Boolean values, objects, and arrays
- Arrays
  - Enclosed in square brackets `[]`
  - Values when top-level: objects
  - Values when used as a value in an object: anything an object can store

Here are some examples of JSON. First, just an object:

```
{
    name: "project1",
    version: "1.0.1",
    author: "jdoe99"
}
```

An array of objects:

```
[
    {
        name: "John Doe",
```

```
        courses: ["CSE233", "CSE235"]
    },
    {
        name: "Jane Doe",
        courses: ["CSE234", "CSE235"]
    }
]
```

An object of arrays:

```
{
    courses: [
        {
            code: "CSE233",
            name: "C++ Programming"
        },
        {
            code: "CSE234",
            name: "Advanced C++ Programming"
        },
        {
            code: "CSE235",
            name: "Python Development"
        }
    ],
    sections: [
        {
            code: "CSE233",
            sectionId: 1,
            instructor: "staff"
        },
        {
            code: "CSE233",
            sectionId: 2,
            instructor: "staff"
        }
    ]
}
```

Objects map to dictionaries in Python, while arrays map to lists. This makes converting JSON data to Python data structures simple. The `json` module will do this for you. The `json.load()` function reads a JSON file. It will return either a dictionary, or a list of dictionaries, depending on the format of the input file. The `json.dump()` function writes a JSON file:

```
import json
```

```
# open the JSON file
with open("input.json") as f:
    # read the data
    json_data = json.loads(f)

# write the json data to another file
with open("output.json", mode="w") as f:
    json.dumpd(f)
```

For the first JSON example, `json.load()` would return a dictionary like the following:

```
{
    'name': "project1",
    'version': "1.0.1",
    'author': "jdoe99"
}
```

For the second JSON example, `json.load()` would return a list of dictionaries:

```
[
    {
        'name': "John Doe",
        'courses': ["CSE233", "CSE235"]
    },
    {
        'name': "Jane Doe",
        'courses': ["CSE234", "CSE235"]
    }
]
```

For the third, it would return a dictionary with lists as its values:

```
{
    'courses': [
        {
            'code': "CSE233",
            'name': "C++ Programming"
        },
        {
            'code': "CSE234",
            'name': "Advanced C++ Programming"
        },
        {
```

```
            'code': "CSE235",
            'name': "Python Development"
        }
    ],
    'sections': [
        {
            'code': "CSE233",
            'sectionId': 1,
            'instructor': "staff"
        },
        {
            'code': "CSE233",
            'sectionId': 2,
            'instructor': "staff"
        }
    ]
}
```

The `json` module also provides member functions called `loads()` and `dumps()`. The `s` stands for strings, and is used to convert strings to JSON (`loads()`), or dictionaries (or a list of dictionaries) to JSON (`dumps()`).

Conclusion

This week, we discussed sets and dictionaries in Python. Sets are collections of elements where every element is unique. Dictionaries store (key, value) pairs. The keys in a dictionary are unique, but the values are not. The elements of sets and keys of dictionaries have to be "hashable". The basic, immutable types (ones that aren't modifiable by functions) in Python are all hashable. JSON is a common data format that is easily represented by Python's lists and dictionaries.
